Clustering Algorithms

7 algorithms · with examples · Jupyter-ready code

Fawad Saleem

01

DENSITY-BASED

DBSCAN

Density-Based Spatial Clustering of Applications with Noise

🔢 No K needed | 🚫 Handles outliers | 🔷 Any shape | ⚡ Fast on large data

THE IDEA

Groups together points that are closely packed. Points in sparse, empty areas are labelled `-1` — meaning they are outliers and don't belong to any cluster. Unlike K-Means, it never forces a point into a group.

KEY PARAMETERS

PARAM	WHAT IT CONTROLS
`eps`	Max distance to be considered a "neighbour." Small = tight clusters. Large = loose clusters.
`min_samples`	Min points needed to form a dense region. Increase to reduce noise labels.

REAL-WORLD EXAMPLES

🌍 Geographic clustering — finding hotspots of crime or disease in a city. Isolated incidents = outliers.

💳 Fraud detection — normal transactions form dense clusters. Fraudulent ones appear as isolated outlier points.

🛰 Satellite imagery — grouping land-use regions. Forest, urban, water naturally form dense regions with irregular shapes.

PYTHON · JUPYTER

COPY

```python
from sklearn.cluster import DBSCAN
import numpy as np

# eps = neighbourhood radius | min_samples = minimum points to form a cluster
db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X)

# label = -1 means OUTLIER — does not belong to any cluster
print(labels)           # e.g. [ 0  0  1  1 -1  0  2 -1]

# Count how many outliers were found
outliers = np.sum(labels == -1)
print(f"Outliers found: {outliers}")
```

02

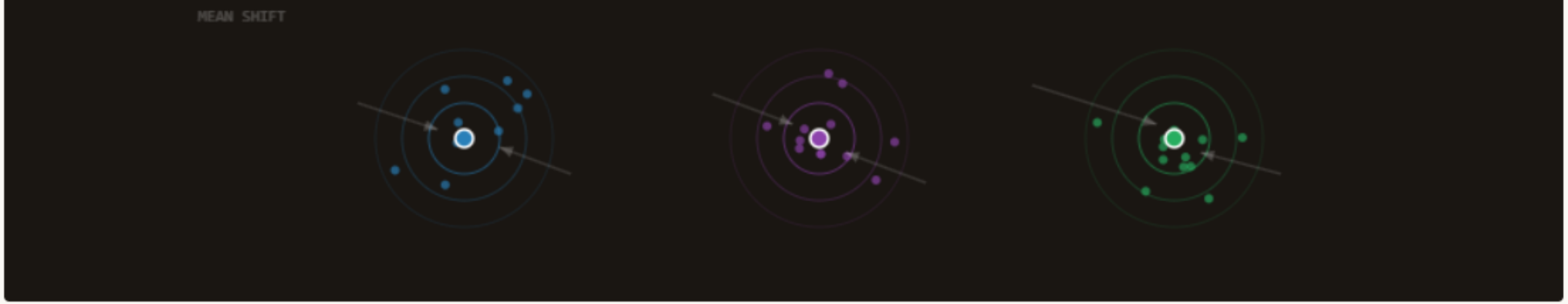
CENTROID-BASED

Mean Shift

Kernel Density Estimation Based Clustering

🔢 No K needed | ⭕ Smooth blobs | 🐢 Slow on large data | 🎯 Deterministic

// MEAN_SHIFT_ANIMATION — CIRCLES ROLLING TOWARD DENSITY PEAKS



THE IDEA

Place a circle on each data point. The circle slides toward the densest nearby area — like a ball rolling toward the lowest point of a bowl. Where all circles settle = cluster centres. The number of clusters is discovered automatically.

KEY PARAMETER

PARAM	WHAT IT CONTROLS
`bandwidth`	Size of the sliding window. Larger = fewer broad clusters. Smaller = more fine-grained clusters. Use `estimate_bandwidth()` to auto-detect.

REAL-WORLD EXAMPLES

📷 Image segmentation — grouping pixels of similar colour together. Each colour cluster = one region of the image.

🖥️ Video tracking — tracking moving objects frame-by-frame. The cluster centre follows the object as it moves.

🗺️ Urban activity mapping — finding natural congregation areas in a city from GPS movement data.

PYTHON · JUPYTER

COPY

```python
from sklearn.cluster import MeanShift, estimate_bandwidth

# Auto-estimate the best bandwidth from your data
bw = estimate_bandwidth(X, quantile=0.2)

ms = MeanShift(bandwidth=bw)
labels = ms.fit_predict(X)

# See how many clusters it found automatically
print(f"Clusters found: {len(set(labels))}")

# Where are the cluster centres?
print(ms.cluster_centers_)
```

03

MESSAGE-PASSING

Affinity Propagation

Exemplar-Based Clustering via Message Passing

🔢 No K needed | 📌 Real data as centres | 🐢 Slow on large data | ✉️ Message-based voting

THE IDEA

Every data point sends messages to all other points: *"I want you as my representative"* or *"I think I should be my own representative."* Points vote back and forth until agreement is reached. Winners become **exemplars** — actual data points that represent their cluster.

KEY PARAMETERS

PARAM	WHAT IT CONTROLS
`damping`	Smooths messages between 0.5–1.0. Higher = more stable, fewer oscillations.
`preference`	More negative = fewer clusters. Less negative = more clusters.

REAL-WORLD EXAMPLES

- 📰 News article clustering — grouping similar news stories. The exemplar = the most representative article in each topic.
- 🧬 Gene expression analysis — finding which genes behave similarly. Cluster centres are actual genes, not invented midpoints.
- 🎵 Music recommendations — grouping songs by similarity. The exemplar = the most "typical" song of that style.

PYTHON · JUPYTER

COPY

```python
from sklearn.cluster import AffinityPropagation

# damping: 0.5-1.0 | preference: more negative = fewer clusters
ap = AffinityPropagation(damping=0.9, preference=-50)
labels = ap.fit_predict(X)

# cluster_centers_indices_ = which original data points are exemplars
print("Exemplar indices:", ap.cluster_centers_indices_)
print(f"Clusters found: {len(ap.cluster_centers_indices_)}")
```

04

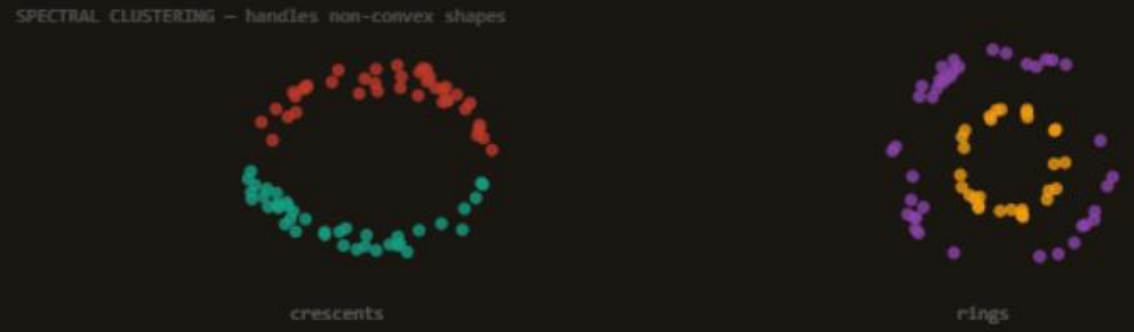
GRAPH-BASED

Spectral Clustering

Clustering via Graph Laplacian Eigenvectors

🔢 Needs K | 🌀 Complex shapes | 🐢 Slow on large data | ⚛ Graph-based

// SPECTRAL_SHAPES — HANDLES NON-CONVEX CLUSTERS K-MEANS CANNOT



THE IDEA

Instead of measuring straight-line distance, it first builds a graph — connecting nearby points with edges. Then it finds groups in that graph. This lets it discover clusters that are tangled, ring-shaped, or crescent-shaped — shapes K-Means completely fails on.

KEY PARAMETERS

PARAM	WHAT IT CONTROLS
`n_clusters`	How many clusters you want. Required upfront.
`affinity`	How the graph is built. `'nearest_neighbors'` or `'rbf'` are common.

REAL-WORLD EXAMPLES

🖼️ Image segmentation — separating objects in a photo that are interleaved or irregular in shape.

🧠 Brain connectivity — finding functional regions in fMRI brain scans where regions are non-convex.

🌐 Community detection — finding groups in social networks where relationships form complex graph structures.

PYTHON · JUPYTER

COPY

```python
from sklearn.cluster import SpectralClustering

# n_clusters required | affinity = how the graph is built
sc = SpectralClustering(
    n_clusters=3,
    affinity='nearest_neighbors',
    random_state=42
)
labels = sc.fit_predict(X)
print(labels)

# Note: SpectralClustering has no .predict() for new data
# It only works on the data it was fitted on
```

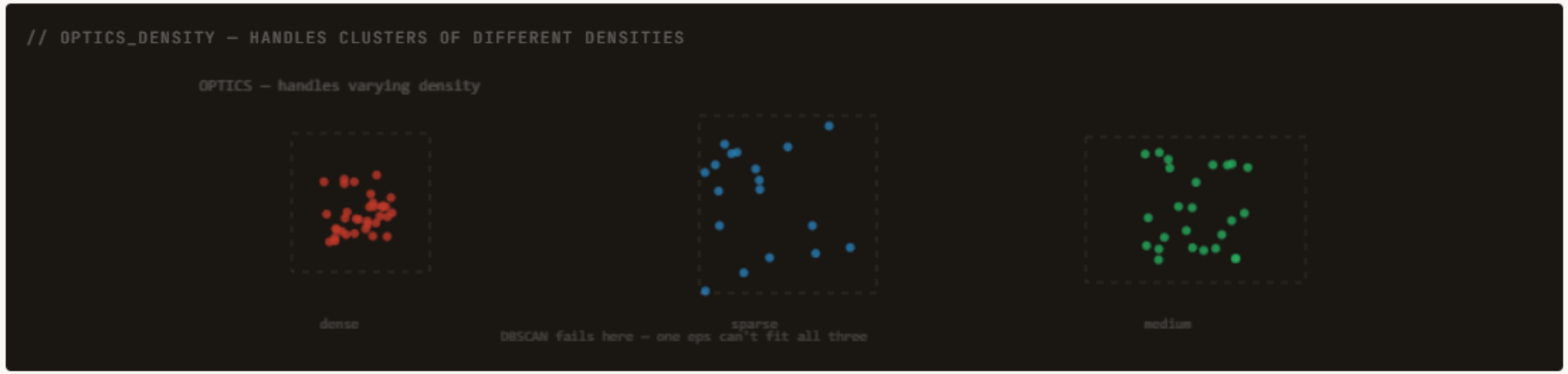
05

DENSITY-BASED · DBSCAN+

OPTICS

Ordering Points To Identify the Clustering Structure

🔢 No K needed | 🚫 Handles outliers | 📊 Variable density | 🔷 Any shape



THE IDEA

DBSCAN's biggest weakness: you set one `eps` for the whole dataset — but what if some regions are denser than others? OPTICS orders all points by **how connected they are**, then extracts clusters at different density levels automatically.

KEY PARAMETER

PARAM	WHAT IT CONTROLS
`min_samples`	Minimum points to form a core region. Same concept as DBSCAN.
`xi`	Steepness of density drops that define cluster boundaries. Default 0.05.

REAL-WORLD EXAMPLES

🌆 **City density analysis** — a city has dense downtown and sparse suburbs. DBSCAN needs different eps for each. OPTICS handles both in one run.

🌊 **Ocean current mapping** — ocean data has dense warm currents and sparse cold regions — varying density throughout.

📡 **Sensor networks** — IoT sensors placed unevenly across a building or region produce data with varying local density.

PYTHON · JUPYTER COPY

```python
from sklearn.cluster import OPTICS

# min_samples = same concept as DBSCAN
op = OPTICS(min_samples=5, xi=0.05)
labels = op.fit_predict(X)

# Same as DBSCAN: label -1 = outlier
print(labels)

# OPTICS also gives you a reachability plot
# which shows density structure visually
print("Outliers:", np.sum(labels == -1))
```

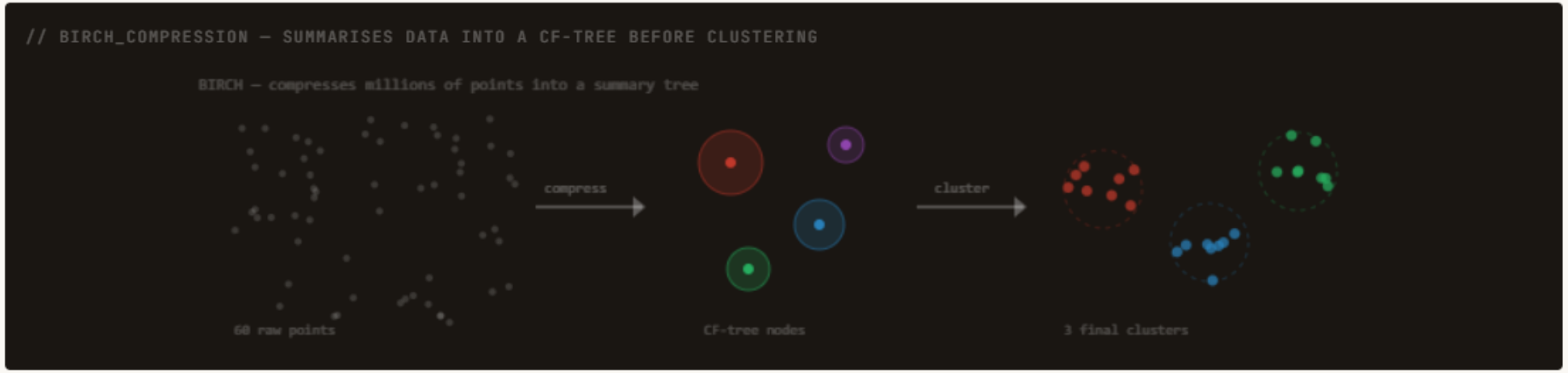
06

HIERARCHICAL · LARGE-SCALE

BIRCH

Balanced Iterative Reducing and Clustering using Hierarchies

🔢 Optional K | 💾 Memory-efficient | ⚡ Fast on huge data | ⭕ Round clusters



THE IDEA

Instead of keeping all data points in memory, BIRCH reads the data once and builds a compressed summary tree (called a CF-Tree). Clustering is then done on this summary — not the full data. This makes it far more efficient than K-Means on millions of rows.

KEY PARAMETERS

PARAM	WHAT IT CONTROLS
`threshold`	Max radius of each summary node. Smaller = more detailed summary.
`n_clusters`	Optional. If set, applies a final clustering step to the summary.

REAL-WORLD EXAMPLES

🛒 E-commerce transactions — clustering millions of purchase records. K-Means would run out of memory. BIRCH handles it efficiently.

📱 Telecom user segmentation — grouping millions of mobile users by usage patterns in real time.

🏦 Banking transactions — processing large transaction logs to group customer spending behaviour at scale.

PYTHON · JUPYTER COPY

```python
from sklearn.cluster import Birch

# threshold = summary node radius | n_clusters = optional final step
birch = Birch(threshold=0.5, n_clusters=3)
labels = birch.fit_predict(X)
print(labels)

# BIRCH also supports partial_fit() for streaming / online data
# birch.partial_fit(X_batch) — processes data in chunks
```